Consilience + Combinatory Creativity MVP

A local, dependency-free prototype for turning a challenge into *traceable cross-domain concept combinations*. It is designed around your Consilience / Combinatory Creativity premise: a **durable memory of ideas, explicit bridges across domains, divergence before convergence, and evaluation before execution.**

What this prototype does

1. Stores concepts as reusable "idea atoms" with a domain, problem, mechanism, principle, affordances, constraints, and abstraction levels.
2. Retrieves relevant atoms using lexical similarity at several abstraction levels—not just literal topic matches.
3. Requires cross-domain pairing, then identifies a shared principle and creates a bridge hypothesis.
4. Generates a concept card with mechanism, MVP test, assumptions, risks, and scores for novelty, value, feasibility, and evidence.
5. Writes an auditable JSON run record so future cycles can retain, challenge, and recombine prior work.

It is intentionally deterministic and local. That makes the system inspectable before you attach a model, web research, vector retrieval, tool execution, or autonomous actions.

Run it

```
python ccc_engine.py "How might we make teams learn from project failures without bla
```

Then inspect `run.json`. No API key or package installation is required.

Add your knowledge

Edit `data/idea_atoms.json`. Keep every atom concrete enough to test and abstract enough to bridge. The fields `principle`, `mechanism`, `affordances`, `constraints`, and `levels` are the important ones.

Upgrade path

- Replace `token_similarity` with embedding retrieval plus graph traversal.
- Add a research/retrieval agent that only adds sources with provenance.
- Add domain specialists, an analogist, a skeptic, and a prototype/test agent.
- Make scoring empirical: evaluate novelty against a corpus, feasibility against constraints/cost, and value with target users.

- Put real-world tools behind permissions, budgets, allow-lists, logs, and human approval.

Non-negotiable design rule

Ideas may be autonomous; consequential actions must be bounded. Generation can be broad. Publishing, spending, changing production systems, contacting people, or running external tools should require explicit policy checks and human approval.